

tp 2026-06-03

May 27, 2026

1 Travaux pratiques - mise en oeuvre de méthodes de prévisions pour les séries temporelles avec algorithmes de gradient boosting

1.1 Déroulement du TP

Ces travaux vous proposent de mettre en oeuvre des techniques de prévision par régression sur de ses valeurs passées et des facteurs exogènes. Nous allons utiliser des algorithmes de gradient boosting.

1.1.1 Développements requis

Des éléments de code vous sont fournis pour préparer les données requises et vous guider / simplifier certains points clés. Vos développements de codes sont à exécuter de préférence dans un notebook Jupyter, mais vous pouvez aussi les faire dans des scripts Python.

Une proposition de corrigé complet reprenant chacune des sections de l'énoncé vous est donnée à la fin. N'hésitez pas à l'utiliser pour vous aider à avancer certains points bloquants.

Au préalable, vous devez installer les bibliothèques requises.

Si vous êtes familier du gestionnaire de dépendances moderne nommé `uv`, vous pouvez utiliser le fichier `pyproject.toml` fourni pour installer les dépendances avec la commande `uv sync`. Un fichier `requirements.txt` est également disponible pour une installation avec `pip install -r requirements.txt`.

1.1.2 Objectif du TP

Ce TP vise à mettre en oeuvre une chaîne de traitements pour prévoir la consommation électrique à H+48 de la Bretagne à partir de données historiques et de facteurs exogènes (météo, vacances, températures prévues à 48h).

La préparation des données vous fournira une série temporelle de consommation. La météo prévisionnelle sera insérée dans les données. La température conditionne fortement la consommation de la Bretagne. Les données de vacances et fériés sont aussi des facteurs importants à prendre en compte, fournis dans les données.

L'objectif est de construire un modèle de prévision de la consommation à l'aide d'algorithmes de gradient boosting, en utilisant les données historiques et les facteurs exogènes (météo) et les vacances scolaires.

Les étapes suivantes vous guideront à travers :

- de préparation de l'échantillon d'apprentissage,
- compréhension des séries temporelles et de leurs caractéristiques,
- construction d'un modèle de benchmark simple,
- de construction du modèle de gradient boosting et sa validation croisée,
- apprentissage du modèle et choix des hyperparamètres,
- d'évaluation des performances et comparaison à un benchmark simple,
- la mise en oeuvre d'éléments d'explicabilité des modèles.

1.2 Préparation de l'échantillon de données

L'ensemble des données vous est facilitée et fournie via une fonction `collecte_et_prepare_donnees_tp()` avec : * la consommations électriques à l'heure de la région Bretagne (source RTE) depuis 2023 jusqu'à maintenant (données live) en MW, * la météo prévisionnelle de température (source : open-meteo.com) à h+48 pour la Bretagne, sur la même historique et jusqu'à H+48 (données live), * les vacances scolaires de la Bretagne pour la zone B (source : data.gouv.fr) sur la même période.

Les données commencent au 1er janvier 2023 et *sont mises à jour en continu* (à chaque exécution `collecte_et_prepare_donnees_tp()`).

La prévision de consommation d'énergie est un cas très courant d'utilisation de séries temporelles avec des facteurs exogènes. Ces prévisions sont utiles (sur des échéances de 24h à 48h) pour la gestion de la production d'électricité, l'optimisation des réseaux, la planification de la maintenance et surtout aussi le trading d'électricité sur les marchés de gros de l'énergie.

Le chargement des données est réalisé par :

```
from preparation_donnees_sources import collecte_et_prepare_donnees_tp
donnees_completes = collecte_et_prepare_donnees_tp()
```

Vous obtenez alors un DataFrame avec les colonnes suivantes :

- **datetime** : la date et l'heure de la mesure (en index Pandas)
- **consommation_elec** : la consommation électrique à l'heure donnée (en MW)
- **temperature** : la température prévisionnelle à l'heure donnée (en °C)
- **vacances** : un indicateur binaire indiquant si l'heure donnée correspond à une période de vacances scolaires (1) ou non (0) en Bretagne.

Toutes les lignes de données nécessaires à la prévision, pour la consommation dans deux jours, sont présentes dans ce DataFrame. Logiquement la variable à prédire **consommation_elec** est disponible jusqu'à maintenant. On ne connaît pas encore la consommation à venir, c'est l'objet du TP. Mais on connaît les prévisions de température et les vacances scolaires jusqu'à H+48.

Pour faciliter le TP, le pas de temps a été régularisé (chaque 1 heure) sur l'ensemble de la période.

1.3 Compréhension des séries temporelles et de leurs caractéristiques

- Proposez ici une visualisation de la série temporelle de consommation électrique, de la température et des vacances scolaires.
- Identifiez visuellement les saisonnalités et données manquantes, de préférence avec la bibliothèque `plotly`, très adaptée à l'exploration des données temporelles. Pour visualiser des séries temporelles avec de nombreux points interactivement, il est recommandé d'utiliser

`go.Scattergl` de `import plotly.graph_objects as go` qui est optimisé pour les grandes séries.

- Evaluer les données manquantes (la série est régulière, chaque pas de temps est présent, mais les valeurs peuvent être manquantes). Que peut-on faire des données manquantes sachant la classe de modèles qu'on utilisera par la suite (méthode d'arbres) ?
- Vérifier les corrélations entre les différentes variables (consommation, température, vacances) et leur évolution dans le temps. Quelles relations semblent présentes ?
- Y-a-t-il de l'autocorrélation pour la variable d'intérêt `consommation_elec` ? présentez des visualisations de l'autocorrélation (ACF) et de l'autocorrélation partielle (PACF) pour la variable `consommation_elec`. Quelles sont les implications de ces autocorrélations pour la construction du modèle de prévision à H+48 ?

1.4 Construction des variables de prévision (features engineering)

- Pour construire un modèle de prévision à H+48, il est nécessaire de construire des variables de prévision (features) à partir des données historiques. Proposez une méthode pour construire ces variables à partir des données disponibles.
- Les variables prédictives souhaitées sont les suivantes :
 - `consommation_elec_h_minus_48` : la consommation électrique à l'heure précédente (H-48)
 - `consommation_elec_mean_week` : la consommation électrique moyenne de la semaine précédente disponible au moment de la prévision (J-8 à J-2, soit H-192 à H-48)
 - `temperature` : la température prévisionnelle à H+48, déjà fournie dans les données
 - `vacances` : l'indicateur binaire de vacances scolaires à H+48, déjà fourni dans les données
 - `hour_in_day` : une variable entière indiquant l'heure de la journée (0 à 23)
 - `day_of_week` : un entier indiquant le jour de la semaine
 - `day_of_year` : un entier indiquant le jour de l'année

Pour les variables `hour_in_day`, `day_of_week` et `day_of_year`, une fonction vous est donnée pour les créer directement à partir du DataFrame et son index.

```
from preparation_donnees_sources import ajouter_indicateurs_temporels
# Ajout des variables temporelles (heure de la journée, jour de la semaine, jour de l'année)
ml_dataset = ajouter_indicateurs_temporels(ml_dataset)
```

- Séparez votre dataset en :
 - un dataset d'apprentissage (*train_data*) avec les données de 2023 jusqu'à il y a 4 mois,
 - un dataset de test (*test_data*). Le dataset de test doit correspondre aux données les plus récentes (4 derniers mois) lorsque les données d'électricité sont présentes,
 - un dataset de prévision (*forecast_data*) seulement pour prévoir de H+1 à H+48 (le modèle de prévision n'a naturellement pas accès à la consommation électrique de ces points de données, mais doit pouvoir utiliser les prévisions de température et les vacances scolaires pour ces points de données).

1.5 Construction d'un modèle de benchmark simple

- Proposez un modèle de benchmark très simple pour la prévision à H+48 de la consommation électrique (de préférence uniquement avec des variables historiques de consommation électrique). Il servira de référence pour évaluer les performances du modèle de gradient boosting que nous allons construire par la suite.
- Vérifiez des propriétés importantes de stationnarité de votre série temporelle de consommation électrique prévue par le modèle simple (conditionnel aux regressseurs) : les résidus du modèle simple de benchmark (différence entre les valeurs prédites et les valeurs réelles) ont-ils une moyenne stable et une absence de dérive significative dans le temps ?

1.6 Création de la validation croisée

Pour identifier les hyperparamètres du modèle de gradient boosting, il est requis de construire une validation croisée. Cependant, la nature temporelle des données doit être prise en compte pour éviter les fuites de données (data leakage de données dans le futurs prises dans l'entraînement du modèle) et obtenir une évaluation réaliste des performances du modèle. Utilisez uniquement 3 folds (pour limiter le temps de calcul).

- Ici nous allons vous vous proposons de construire un premier type de validation croisée qui nie la dimension temporelle des données : un simple KFold (données aléatoirement tirées entre *train* et *test*). Arrêtez vous à la création des folds de validation croisée, sans faire l'apprentissage du modèle. Visualisez les dates des points de données retenus pour le *train* et le *test* de vos folds (à minima pour un Fold). Pourquoi ce graphique montre que la dimension temporelle n'est pas respectée ? Quelles conséquences cela peut-il avoir sur l'évaluation des performances du modèle ?
- Proposez une validation croisée respectant la dimension temporelle des données (par ex. `TimeSeriesSplit` de scikit-learn). Visualisez les dates des points de données retenus pour le *train* et le *test* de vos folds (à minima pour un Fold). Pourquoi ce graphique montre que la dimension temporelle est respectée ?

1.7 Construction du modèle de gradient boosting et choix des hyperparamètres

- Construisez un modèle de gradient boosting pour la prévision à H+48 de la consommation électrique à l'aide de la bibliothèque `lightgbm`.
- de regression (car la variable à prévoir est continue) avec un gradient boosting (`lightgbm`),
- nous vous proposons d'optimiser uniquement les hyperparamètres suivants :
 - `n_estimators` : taille de la séquence d'arbres du modèle de gradient boosting,
 - `learning_rate` : le taux d'apprentissage du modèle de gradient boosting, qui contrôle la contribution de chaque arbre à la séquence d'arbres,
 - `max_depth` : la profondeur maximale des arbres de décision dans le modèle de gradient boosting, qui contrôle la complexité des arbres et peut aider à prévenir le surapprentissage,
 - `num_leaves` : le nombre maximum de feuilles dans les arbres de décision du modèle de gradient boosting, qui contrôle la complexité des arbres et peut aider à prévenir le surapprentissage.

Une grille possible d’hyper-paramètres à tester est la suivante :

```
grid = {  
    "n_estimators": [50, 100, 300, 600],  
    "learning_rate": [0.01, 0.03, 0.05],  
    "num_leaves": [15, 31, 63, 127],  
    "max_depth": [-1, 6, 10, 15],  
}
```

- La recherche d’hyperparamètres est une étape clé pour optimiser les performances du modèle. Proposez une méthode de recherche d’hyperparamètres principaux (par ex. `GridSearchCV` ou `RandomizedSearchCV` de `scikit-learn`) en utilisant la validation croisée respectant la dimension temporelle que vous avez construite précédemment.
- (optionnel, si le temps le permet) Les méthodes de recherche d’hyperparamètres peuvent être très coûteuses en temps de calcul si les grilles sont larges, à pas fins et le nombre d’hyperparamètres à tester est élevé. Est-ce que faire des tests d’ensembles de valeurs indépendants est une bonne idée, quand on sait qu’on a déjà testé des valeurs pour d’autres hyperparamètres proches ? Ici une méthode de recherche d’hyperparamètres plus efficace est recommandée, adoptant une approche bayésienne : “Étant donné les essais déjà observés, où est-il le plus probable de trouver de bonnes performances ?”
 - Utilisez la bibliothèque `optuna` pour implémenter cette méthode de recherche d’hyperparamètres.
 - Comparez les performances du modèle de gradient boosting avec les hyperparamètres trouvés par la recherche aléatoire et par la recherche bayésienne sur la moyenne des échantillons de validation croisée.
 - Comparez la performance de ces deux modèles sur l’échantillon de test. Quelle stratégie de recherche d’hyperparamètres semble la plus efficace pour ce cas d’utilisation ?

1.8 Evaluer votre modèle et prévoyons

- Sur l’échantillon de test, évaluez les performances de votre modèle de gradient boosting et comparez-les à celles du modèle de benchmark simple que vous avez construit précédemment.
- Prévoyez la consommation électrique à H+48 pour les prochaines heures (jusqu’à H+48) et visualisez vos prévisions.

1.9 (optionnel, si le temps le permet) Explicabilité du modèle et importance des regressseurs

Pour comprendre le comportement du modèle de gradient boosting appris et permettre aux utilisateurs de mieux s’appropriier l’outil, proposez une analyse de l’importance des regressseurs dans le modèle à l’aide de la méthode des *Shapley values*. Cette méthode présente des propriétés intéressantes pour l’explicabilité des modèles d’apprentissage automatique, notamment la possibilité d’obtenir des explications globales (importance moyenne de chaque variable dans le modèle) et locales (contribution de chaque variable à la prédiction pour une observation donnée).

Les *Shapley values* présentent des propriétés supérieures à la plupart des autres méthodes d’explicabilités (efficacité, symétrie, additivité, nullité, ...) et sont basées sur une théorie de jeux coopératifs.

Fournissez ces explications : * 1/ globales * 2/ locales pour la dernière observation de l'échantillon de prévision à l'aide de la bibliothèque `shap`.

1.10 Proposition de corrigé

1.10.1 Préparation des données

```
[1]: from preparation_donnees_sources import collecte_et_prepare_donnees_tp
donnees_completes = collecte_et_prepare_donnees_tp()
```

Le format des données est le suivant :

```
[2]: donnees_completes
```

```
[2]:
```

	consommation_elec	temperature	vacances
datetime			
2023-01-01 00:00:00+01:00	2522.0	11.1	True
2023-01-01 01:00:00+01:00	2505.0	10.8	True
2023-01-01 02:00:00+01:00	2514.0	10.6	True
2023-01-01 03:00:00+01:00	2229.0	10.6	True
2023-01-01 04:00:00+01:00	2049.0	10.5	True
...	...	...	...
2026-05-29 09:00:00+02:00	NaN	19.6	False
2026-05-29 10:00:00+02:00	NaN	21.2	False
2026-05-29 11:00:00+02:00	NaN	23.2	False
2026-05-29 12:00:00+02:00	NaN	25.0	False
2026-05-29 13:00:00+02:00	NaN	26.5	False

[29869 rows x 3 columns]

1.10.2 Compréhension des séries temporelles et de leurs caractéristiques

1/ Dynamiques des variables dans le temps

```
[3]: from plotly.subplots import make_subplots
import plotly.graph_objects as go
# Graphs avec trois subplots (trois lignes, une pour la consommation, une pour
↳ la température et une pour les vacances)

fig = make_subplots(rows=3, cols=1, shared_xaxes=True, vertical_spacing=0.1)

markers_size = dict(size=3)

# Consommation électrique
fig.add_trace(
    go.Scattergl(
        x=donnees_completes.index, y=donnees_completes["consommation_elec"],
        mode='markers',
        name="Consommation électrique (MWh)",
```

```

        marker=markers_size
    ),
    row=1, col=1,
)
# Température
fig.add_trace(
    go.Scattergl(
        x=donnees_completes.index, y=donnees_completes["temperature"],
        mode='markers',
        name="Température (°C)",
        marker=markers_size
    ),
    row=2, col=1
)

fig.add_trace(
    go.Scattergl(
        x=donnees_completes.index, y=donnees_completes["vacances"],
        mode='markers',
        name="Vacances scolaires",
        marker=markers_size
    ),
    row=3, col=1
)

fig.update_layout(height=800, width=1000, title_text="Consommation électrique_
↳(MW) et température au fil du temps (°C)")
fig.show()

```

2/ Suivi des données manquantes

```
[4]: donnees_completes.isna().sum()
```

```
[4]: consommation_elec    48
temperature                516
vacances                   0
dtype: int64
```

```
[5]: donnees_completes[donnees_completes["temperature"].isna()]
```

```
[5]:
```

	consommation_elec	temperature	vacances
datetime			
2023-12-30 01:00:00+01:00	2610.0	NaN	True
2023-12-30 02:00:00+01:00	2646.0	NaN	True
2023-12-30 03:00:00+01:00	2483.0	NaN	True
2023-12-30 04:00:00+01:00	2314.0	NaN	True
2023-12-30 05:00:00+01:00	2259.0	NaN	True
...	...	...	...

2024-01-20 08:00:00+01:00	4104.0	NaN	False
2024-01-20 09:00:00+01:00	4312.0	NaN	False
2024-01-20 10:00:00+01:00	4378.0	NaN	False
2024-01-20 11:00:00+01:00	4245.0	NaN	False
2024-01-20 12:00:00+01:00	4072.0	NaN	False

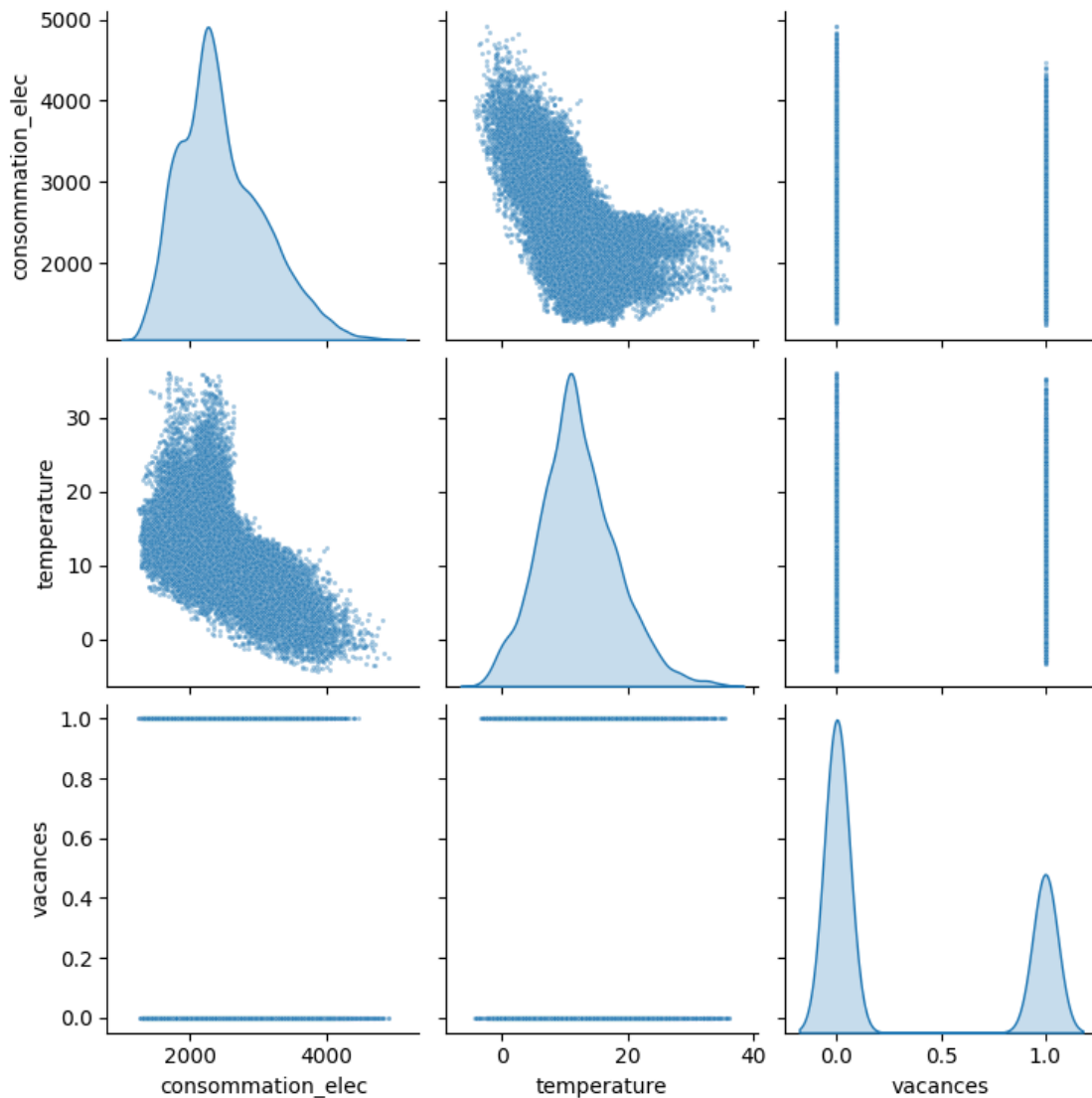
[516 rows x 3 columns]

Certaines données de température manquent, mais pas sur la fin de la série, nécessaire à la prévision.

1.10.3 Analyses sommaire des données

```
[6]: # Compute pairwise correlation plots
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(donnees_completes, kind="scatter", diag_kind="kde", plot_kws={
    "alpha": 0.4,
    "s": 5
})
plt.show()
```

Il s'agit de densités sur la diagonale. Les modalités discrètes sont lissées, alors qu'en réalité elles sont discrètes.

On vérifie que la grille temporelle est régulière et complète (aucun pas de temps manquant).

```
[7]: print(donnees_completes.index.is_unique)
```

True

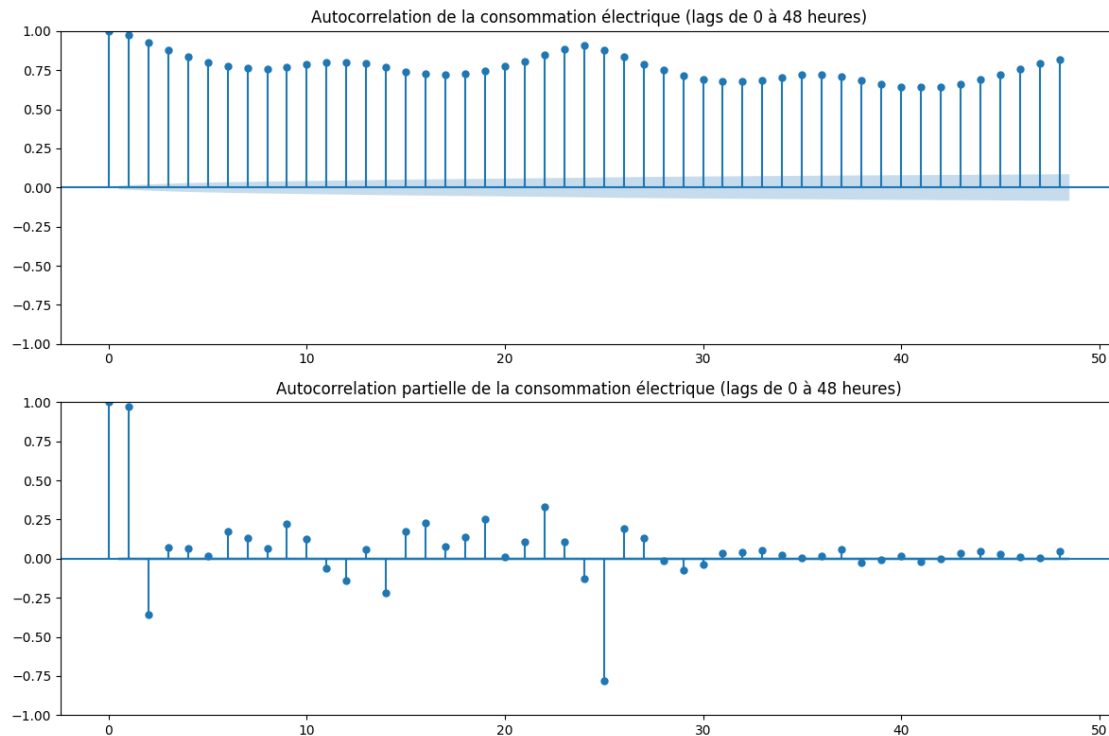
Les autocorrélogrammes et autocorrélogrammes partiels sont les suivants :

```
[8]: from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
fig, axes = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(donnees_completes["consommation_elec"].dropna(), lags=48, ax=axes[0])
plot_pacf(donnees_completes["consommation_elec"].dropna(), lags=48, ax=axes[1])
```

```

axes[0].set_title("Autocorrelation de la consommation électrique (lags de 0 à 48 heures)
↪48 heures)")
axes[1].set_title("Autocorrelation partielle de la consommation électrique
↪(lags de 0 à 48 heures)")
plt.tight_layout()
plt.show()

```



1.10.4 Préparation des variables des modèles de prévision

```

[9]: from pandas import Timedelta

from preparation_donnees_sources import ajouter_indicateurs_temporels

# Copy data to avoid touching the original dataframe and make sure it is sorted
↪by date (index)
ml_dataset = donnees_completes.copy().sort_index()

# * `consommation_elec_h_minus_48` : la consommation électrique à l'heure
↪précédente (H-48). On controle la date de la consommation à H-48.
ml_dataset["consommation_elec_h_minus_48"] = ml_dataset["consommation_elec"].
↪shift(48)      # Cela décale vers le bas les heures (ordonnées de la plus
↪récente à la plus ancienne)
ml_dataset["datetime"] = ml_dataset.index

```

```

ml_dataset["datetime_minus_48h"] = ml_dataset.index + Timedelta(hours=-48)

# * `consommation_elec_mean_week` : la consommation électrique moyenne de la
↳ semaine précédente disponible au moment de la prévision (J-8 à J-2, soit
↳ H-192 à H-48).
# On se base sur la consommation électrique à H-48 pour calculer la moyenne de
↳ la semaine précédente, afin de ne pas introduire de fuite de données. et on
↳ applique une moyenne glissante sur 7 jours (7*48 = 336 heures) pour calculer
↳ la consommation électrique moyenne de la semaine précédente.
ml_dataset["consommation_elec_mean_week"] =
↳ ml_dataset["consommation_elec_h_minus_48"].rolling(window=24*7).mean()

# Ajout des variables temporelles (heure de la journée, jour de la semaine,
↳ jour de l'année)
ml_dataset = ajouter_indicateurs_temporels(ml_dataset)

# Séparation du dataset en un dataset d'entraînement (train) et un dataset de
↳ test (test) : 4 mois de données les plus récentes pour le test, le reste
↳ pour l'entraînement
debut_test = ml_dataset.index.max() - Timedelta(days=30 * 4)
colonnes_a_supprimer = ["datetime", "datetime_minus_48h"]
# Tout l'échantillon sauf les 4 derniers mois pour l'entraînement (train)
train_data = ml_dataset[ml_dataset.index < debut_test].
↳ drop(columns=colonnes_a_supprimer)
# Les 4 derniers mois pour le test (test) si la consommation électrique est non
↳ manquantes, sinon pour la prévision (forecast)
test_data = ml_dataset[(ml_dataset.index >= debut_test) &
↳ (ml_dataset["consommation_elec"].notna())].drop(columns=colonnes_a_supprimer)
# 48 dernières heures pour la prévision (forecast), avec la consommation
↳ électrique est manquante
forecast_data = ml_dataset[(ml_dataset.index >= debut_test) &
↳ (ml_dataset["consommation_elec"].isna())].drop(columns=colonnes_a_supprimer)

# Séparation en trois datasets : X_train, y_train, X_test, y_test et
y_var = "consommation_elec"
X_train = train_data.drop(columns=[y_var])
y_train = train_data[y_var]
X_test = test_data.drop(columns=[y_var])
y_test = test_data[y_var]
X_forecast = forecast_data.drop(columns=[y_var])

```

1.10.5 Construction d'un modèle de benchmark simple

Ici nous proposons simplement de prendre la consommation à H-48 comme prévision à H+48. C'est un modèle de benchmark très simple, mais qui peut être difficile à battre pour des prévisions à court terme de séries temporelles avec une forte autocorrelation. Il fonctionne bien car la consommation d'électricité est très sensible au moment de l'année (chauffage en hiver) et au moment du jour (pics

au petit-déjeuner et autour du dîner).

Evaluons la performance de ce modèle de benchmark simple à l'aide d'une métrique d'erreur - ici une MAE (Mean Absolute Error, mesure symétrique et qui pénalise également les erreurs positives et négatives) - sur le dataset de test.

```
[10]: print(f"MAE sur benchmark simple : {(test_data['consommation_elec'] -  
      ↪test_data['consommation_elec_h_minus_48']).abs().mean():.2f} MWh en regard  
      ↪de la consommation moyenne de {test_data['consommation_elec'].mean():.2f}  
      ↪MWh")
```

MAE sur benchmark simple : 269.52 MWh en regard de la consommation moyenne de 2295.96 MWh

Nous testons des propriétés de stationnarité des résidus du modèle simple (différence entre les valeurs prédites et les valeurs réelles) : les résidus du modèle simple ont-ils une moyenne stable et une absence de dérive significative dans le temps ? On peut utiliser un test de stationnarité comme le test d'Augmented Dickey-Fuller (ADF) pour vérifier cela.

```
[11]: from statsmodels.tsa.stattools import adfuller  
      from pprint import pprint  
  
      res = adfuller((y_test - X_test["consommation_elec_h_minus_48"]).values)  
  
      print(  
          f"ADF statistic: {res[0]:.4f}\n"  
          f"p-value: {res[1]:.3e}\n"  
          f"used lags: {res[2]}\n"  
          f"nobs: {res[3]}\n"  
          f"icbest: {res[5]:.4f}"  
      )  
      print("critical values:")  
      for level, value in res[4].items():  
          print(f"  {level}: {value:.4f}")
```

ADF statistic: -7.3316
p-value: 1.124e-10
used lags: 25
nobs: 2807
icbest: 33300.1626
critical values:
 1%: -3.4327
 5%: -2.8626
 10%: -2.5673

La lecture du test indique avec la p-value de faible valeur qu'on a suffisamment d'évidence statistique contre l'hypothèse H_0 .

La série n'a probablement pas de racine unitaire, ce qui suggère que les résidus du modèle simple sont stationnaires. Cela signifie que les résidus ont une moyenne stable et une absence de dérive significative dans le temps. Aussi un modèle plus précis (autres facteurs exogènes et meilleurs

algorithmes), les nouveaux résidus hériteront des mêmes propriétés de stationnarité (conditionnelle aux régresseurs), ce qui est une bonne chose pour la prévision.

1.10.6 Création de la validation croisée

- Validation croisée agnostique à la dimension temporelle (KFold)

```
[12]: import plotly.graph_objects as go
      from sklearn.model_selection import KFold

      # Shuffled 4-fold CV
      kf = KFold(
          n_splits=3,
          shuffle=True,
          random_state=0
      )

      # Application sur l'échantillon d'entraînement
      folds = list(kf.split(train_data.reset_index()))

      # Visualisation des date de train et test des folds
      fig = go.Figure()
      for i, (train_index, test_index) in enumerate(folds):
          fig.add_trace(go.Scattergl(
              x=train_data.index[train_index],
              y=[i + 1] * len(train_index),
              mode='markers',
              name=f'Fold {i+1} - Train',
              marker=dict(color='blue', size=5, symbol='circle')
          ))
          fig.add_trace(go.Scattergl(
              x=train_data.index[test_index],
              y=[i + 1] * len(test_index),
              mode='markers',
              name=f'Fold {i+1} - Validation',
              marker=dict(color='red', size=5, symbol='diamond')
          ))

      # Layout
      fig.update_layout(
          title="Shuffled KFold on Time Series",
          xaxis_title="Date",
          yaxis=dict(
              title="Numéro du Fold",
              zeroline=False
          ),
          height=600,
```

```

        width=1000
    )

    fig.update_xaxes(
        rangeselector=dict(
            buttons=[
                dict(step="all"),
                dict(count=6, label="6m", step="month", stepmode="backward"),
            ]
        ),
        type="date",
        range=[train_data.index[train_index].max() - Timedelta(days=14), train_data.
↪index[train_index].max() + Timedelta(days=2)]
    )

    fig.show()

```

Ci-dessus , on observe que les données de train et de test sont mélangées dans le temps. Cela nie la non-échangeabilité des données temporelles, c'est à dire que les données ne sont pas indépendantes dans le temps et identiquement distribuées (i.i.d.) dans le temps.

En mélangeant les données de train et de test dans le temps, des points dans les données d'apprentissages vont être consécutifs à des points de validation. Les dynamiques des points du `train` vont artificiellement influencer des points de validation. Dans l'usage en prévision, l'échantillon de points à prévoir ne sera pas entourés de points de l'apprentissage. Il y a dans ce schéma des cross-validation des fuites des données (data leakage) où des informations du futur (par rapport à l'échantillon de validation) sont utilisées pour entraîner le modèle, ce qui conduit à l'apprentissage d'un modèle inadapté par rapport aux conditions réelles de prévision. Aussi cela génère une évaluation trop optimiste des performances du modèle sur le dataset de test.

```

[13]: import plotly.graph_objects as go
      from sklearn.model_selection import TimeSeriesSplit

      # TimeSeriesSplit 3-fold CV
      timeseriesplit = TimeSeriesSplit(
          n_splits=3,
      )

      # Application sur l'échantillon d'entraînement
      folds = list(timeseriesplit.split(train_data.reset_index()))

      # Visualisation des date de train et test des folds
      fig = go.Figure()
      for i, (train_index, test_index) in enumerate(folds):
          fig.add_trace(go.Scattergl(
              x=train_data.index[train_index],
              y=[i + 1] * len(train_index),

```

```

        mode='markers',
        name=f'Fold {i+1} - Train',
        marker=dict(color='blue', size=3)
    ))
    fig.add_trace(go.Scattergl(
        x=train_data.index[test_index],
        y=[i + 1] * len(test_index),
        mode='markers',
        name=f'Fold {i+1} - Validation',
        marker=dict(color='red', size=3)
    ))

# Layout
fig.update_layout(
    title="TimeSeriesSplit on Time Series",
    xaxis_title="Date",
    yaxis=dict(
        title="Numéro du Fold",
        zeroline=False
    ),
    height=600,
    width=1000
)

fig.show()

```

1.10.7 Définition du modèle et choix des hyperparamètres

Nous construisons ici un modèle : * de regression (car la variable à prévoir est continue) avec un gradient boosting (`lightgbm`), * comme il s'agit d'arbres de décision, les données manquantes ne posent pas de problème pour l'apprentissage du modèle, il n'est pas nécessaire de les imputer ou de les supprimer, * les hyperparamètres optimisés sont : * `n_estimators` : taille de la séquence d'arbres du modèle de gradient boosting, * `learning_rate` : le taux d'apprentissage du modèle de gradient boosting, qui contrôle la contribution de chaque arbre à la séquence d'arbres, * `max_depth` : la profondeur maximale des arbres de décision dans le modèle de gradient boosting, qui contrôle la complexité des arbres et peut aider à prévenir le surapprentissage, * `num_leaves` : le nombre maximum de feuilles dans les arbres de décision du modèle de gradient boosting, qui contrôle la complexité des arbres et peut aider à prévenir le surapprentissage.

Avec 4 hyperparametres, 4 pas pour chaque et 3 folds, cela fait $4^4 * 3 = 768$ modèles à entraîner si un `GridSearchCV` était utilisé, ce qui est déjà assez coûteux en temps de calcul. On utilise ici une recherche aléatoire (`RandomizedSearchCV`) avec 30 ensembles testés.

```

[14]: from lightgbm import LGBMRegressor
      from sklearn.model_selection import RandomizedSearchCV

      # Définition du regresseur
      model = LGBMRegressor(random_state=0, verbose=-1, force_col_wise=True)

```

```

grid = {
    "n_estimators": [100, 300, 600, 1000],
    "learning_rate": [0.01, 0.03, 0.05],
    "num_leaves": [5, 15, 31, 63],
    "max_depth": [-1, 6, 15, 30],
}

# Recherche d'hyperparamètres avec RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=grid,
    scoring="neg_mean_absolute_error",
    cv=timeseriesplit,
    n_jobs=-1,
    verbose=2,
    n_iter=30, # Number of parameter settings that are sampled
    random_state=0
)

# Lancement de la recherche d'hyperparamètres et tuning du meilleur modèle sur
↳ l'échantillon d'entraînement complet
random_search.fit(X_train, y_train)

```

Fitting 3 folds for each of 30 candidates, totalling 90 fits

```

[CV] END learning_rate=0.03, max_depth=15, n_estimators=100, num_leaves=15;
total time= 0.5s
[CV] END learning_rate=0.03, max_depth=15, n_estimators=100, num_leaves=15;
total time= 0.8s
[CV] END learning_rate=0.03, max_depth=15, n_estimators=100, num_leaves=15;
total time= 1.0s
[CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=63;
total time= 3.4s
[CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=63;
total time= 3.8s
[CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=63;
total time= 4.2s
[CV] END learning_rate=0.03, max_depth=-1, n_estimators=600, num_leaves=31;
total time= 12.1s
[CV] END learning_rate=0.03, max_depth=-1, n_estimators=600, num_leaves=31;
total time= 12.3s
[CV] END learning_rate=0.03, max_depth=-1, n_estimators=600, num_leaves=31;
total time= 12.9s
[CV] END learning_rate=0.01, max_depth=6, n_estimators=100, num_leaves=31; total
time= 2.0s
[CV] END learning_rate=0.01, max_depth=6, n_estimators=100, num_leaves=31; total
time= 2.3s
[CV] END learning_rate=0.01, max_depth=6, n_estimators=100, num_leaves=31; total
time= 2.4s

```


[CV] END learning_rate=0.03, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 12.4s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=100, num_leaves=15; total
 time= 1.0s
 [CV] END learning_rate=0.03, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 13.2s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=100, num_leaves=15; total
 time= 1.3s
 [CV] END learning_rate=0.03, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 13.6s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=100, num_leaves=15; total
 time= 1.2s
 [CV] END learning_rate=0.03, max_depth=15, n_estimators=1000, num_leaves=31;
 total time= 19.7s
 [CV] END learning_rate=0.03, max_depth=15, n_estimators=1000, num_leaves=31;
 total time= 20.8s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=1000, num_leaves=31;
 total time= 20.6s
 [CV] END learning_rate=0.03, max_depth=15, n_estimators=1000, num_leaves=31;
 total time= 21.8s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=1000, num_leaves=31;
 total time= 21.7s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=15;
 total time= 3.2s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=1000, num_leaves=31;
 total time= 22.0s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=15;
 total time= 9.6s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=15;
 total time= 3.5s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=15;
 total time= 4.0s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=15;
 total time= 10.8s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=15;
 total time= 10.9s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 13.1s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 13.0s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=5; total
 time= 0.2s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=5; total
 time= 0.3s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=5; total
 time= 0.3s
 [CV] END learning_rate=0.01, max_depth=-1, n_estimators=300, num_leaves=63;
 total time= 13.4s

[CV] END learning_rate=0.05, max_depth=15, n_estimators=600, num_leaves=31;
 total time= 9.0s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=600, num_leaves=31;
 total time= 8.7s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=600, num_leaves=31;
 total time= 9.8s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=31;
 total time= 11.9s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=300, num_leaves=15;
 total time= 2.1s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=31;
 total time= 13.0s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=1000, num_leaves=31;
 total time= 13.0s
 [CV] END learning_rate=0.03, max_depth=-1, n_estimators=100, num_leaves=31;
 total time= 2.0s
 [CV] END learning_rate=0.03, max_depth=-1, n_estimators=100, num_leaves=31;
 total time= 2.2s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=300, num_leaves=15;
 total time= 3.2s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=300, num_leaves=15;
 total time= 3.6s
 [CV] END learning_rate=0.03, max_depth=-1, n_estimators=100, num_leaves=31;
 total time= 2.3s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=300, num_leaves=63;
 total time= 8.3s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=300, num_leaves=63;
 total time= 8.5s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=63;
 total time= 18.4s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=300, num_leaves=63;
 total time= 9.4s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=63;
 total time= 19.2s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=600, num_leaves=5; total
 time= 2.3s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=600, num_leaves=5; total
 time= 2.6s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=600, num_leaves=5; total
 time= 2.7s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=63;
 total time= 20.4s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=5; total
 time= 2.1s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=5; total
 time= 2.3s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=5; total
 time= 2.4s

[CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=31;
 total time= 10.1s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=31;
 total time= 10.6s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=600, num_leaves=31;
 total time= 9.6s
 [CV] END learning_rate=0.05, max_depth=-1, n_estimators=600, num_leaves=31;
 total time= 9.9s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=300, num_leaves=31;
 total time= 3.7s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=600, num_leaves=31;
 total time= 10.6s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=300, num_leaves=31;
 total time= 4.1s
 [CV] END learning_rate=0.03, max_depth=30, n_estimators=600, num_leaves=31;
 total time= 10.7s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=600, num_leaves=5; total
 time= 2.2s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=300, num_leaves=31;
 total time= 4.6s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=600, num_leaves=5; total
 time= 2.4s
 [CV] END learning_rate=0.05, max_depth=6, n_estimators=600, num_leaves=5; total
 time= 2.5s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=5;
 total time= 3.4s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.5s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=5;
 total time= 4.0s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.6s
 [CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=5;
 total time= 3.7s
 [CV] END learning_rate=0.05, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.7s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.6s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.5s
 [CV] END learning_rate=0.01, max_depth=15, n_estimators=100, num_leaves=15;
 total time= 0.9s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=600, num_leaves=63;
 total time= 16.8s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=600, num_leaves=63;
 total time= 16.8s
 [CV] END learning_rate=0.05, max_depth=30, n_estimators=600, num_leaves=63;
 total time= 17.8s

```
[CV] END learning_rate=0.05, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 23.3s
[CV] END learning_rate=0.05, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 23.6s
[CV] END learning_rate=0.05, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 24.3s
[CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 23.3s
[CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 23.4s
[CV] END learning_rate=0.01, max_depth=30, n_estimators=1000, num_leaves=63;
total time= 22.9s
```

```
[14]: RandomizedSearchCV(cv=TimeSeriesSplit(gap=0, max_train_size=None, n_splits=3,
test_size=None),
                        estimator=LGBMRegressor(force_col_wise=True, random_state=0,
                                                verbose=-1),
                        n_iter=30, n_jobs=-1,
                        param_distributions={'learning_rate': [0.01, 0.03, 0.05],
                                           'max_depth': [-1, 6, 15, 30],
                                           'n_estimators': [100, 300, 600, 1000],
                                           'num_leaves': [5, 15, 31, 63]},
                        random_state=0, scoring='neg_mean_absolute_error',
                        verbose=2)
```

En complément, nous proposons une méthode de recherche d'hyperparamètres plus efficace, adoptant une approche bayésienne. Cette méthode est notamment implémentée dans la bibliothèque `optuna`. Prenons le même nombre d'ensemble de paramètres testés (30) que pour la recherche aléatoire, mais avec une approche bayésienne, on peut espérer trouver des hyperparamètres plus performants que la recherche aléatoire, car elle utilise les résultats des essais précédents pour orienter la recherche vers des régions de l'espace d'hyperparamètres plus prometteuses.

```
[15]: from optuna import create_study
from sklearn.model_selection import cross_val_score

# init d'une recherche
study = create_study(direction="minimize")
# Define hyperparameters to tune
def objective(trial):

    # Axes des HP à tester, plages de valeurs et échelle de recherche (linéaire
    ↪ ou logarithmique)
    iterations = trial.suggest_int("n_estimators", 60, 400, log=True)
    learning_rate = trial.suggest_float("learning_rate", 0.02, 0.2, log=True)
    num_leaves = trial.suggest_int("num_leaves", 5, 63, log=True)
    max_depth = trial.suggest_int("max_depth", 5, 100, log=True)

    l2_leaf_reg = trial.suggest_int("l2_leaf_reg", 1, 20, log=True)
```

```

# Prepare the regressor with the parameters suggested by the trial
reg = LGBMRegressor(
    n_estimators=iterations,
    max_depth=max_depth,
    num_leaves=num_leaves,
    learning_rate=learning_rate,
    reg_lambda=l2_leaf_reg,
    random_state=0,
    verbose=-1,
    force_col_wise=True,
    loss_function="mae"

)

# Evaluate the regressor using cross-validation and return the mean score
cv_score = cross_val_score(
    reg,
    X_train,
    y_train,
    cv=timeseriesplit,
    scoring="neg_mean_absolute_error",
    n_jobs=-1,
).mean()

return -cv_score

# Launch the optimization
study.optimize(objective, n_trials=30, show_progress_bar=True)

# Train on the best hyperparameters found on the full training set
best_params = study.best_params
optuna_search = LGBMRegressor(
    n_estimators=best_params["n_estimators"],
    max_depth=best_params["max_depth"],
    num_leaves=best_params["num_leaves"],
    learning_rate=best_params["learning_rate"],
    reg_lambda=best_params["l2_leaf_reg"],
    random_state=0,
    verbose=-1,
    force_col_wise=True,
    loss_function="mae"
)

# Apprentissage sur le dataset d'entraînement complet avec les meilleurs
↳ hyperparamètres trouvés
optuna_search.fit(X_train, y_train)

```

```
/home/julien-gooris/Projects/teaching/2026-06-03 UBS - Sciences des
données/TP/.venv/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning:
IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user\_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
[I 2026-05-27 14:06:05,766] A new study created in memory with name:
no-name-7f6c992c-a01e-4c38-bee8-a75190adf8f6
```

```
Best trial: 0. Best value: 171.981:   3%|          | 1/30 [00:02<01:05,
2.27s/it]
```

```
[I 2026-05-27 14:06:08,033] Trial 0 finished with value:
171.98091858575518 and parameters: {'n_estimators': 213, 'learning_rate':
0.1661426266871959, 'num_leaves': 57, 'max_depth': 15, 'l2_leaf_reg': 18}. Best
is trial 0 with value: 171.98091858575518.
```

```
Best trial: 1. Best value: 167.357:   7%|          | 2/30 [00:04<00:55,
1.99s/it]
```

```
[I 2026-05-27 14:06:09,835] Trial 1 finished with value:
167.35693433739735 and parameters: {'n_estimators': 316, 'learning_rate':
0.04213205005436149, 'num_leaves': 28, 'max_depth': 68, 'l2_leaf_reg': 5}. Best
is trial 1 with value: 167.35693433739735.
```

```
Best trial: 2. Best value: 164.219:  10%|         | 3/30 [00:04<00:33,
1.23s/it]
```

```
[I 2026-05-27 14:06:10,164] Trial 2 finished with value:
164.21947370883774 and parameters: {'n_estimators': 103, 'learning_rate':
0.08616967952557171, 'num_leaves': 15, 'max_depth': 6, 'l2_leaf_reg': 1}. Best
is trial 2 with value: 164.21947370883774.
```

```
Best trial: 2. Best value: 164.219:  13%|         | 4/30 [00:04<00:21,
1.21it/s]
```

```
[I 2026-05-27 14:06:10,371] Trial 3 finished with value:
176.01838521465308 and parameters: {'n_estimators': 75, 'learning_rate':
0.043649556134115274, 'num_leaves': 11, 'max_depth': 29, 'l2_leaf_reg': 17}.
Best is trial 2 with value: 164.21947370883774.
```

```
Best trial: 2. Best value: 164.219:  17%|         | 5/30 [00:05<00:25,
1.03s/it]
```

```
[I 2026-05-27 14:06:11,766] Trial 4 finished with value:
166.1789862638525 and parameters: {'n_estimators': 149, 'learning_rate':
0.06001939263873837, 'num_leaves': 50, 'max_depth': 19, 'l2_leaf_reg': 1}. Best
is trial 2 with value: 164.21947370883774.
```

```
Best trial: 5. Best value: 162.084:  20%|         | 6/30 [00:06<00:21,
1.09it/s]
```

```
[I 2026-05-27 14:06:12,448] Trial 5 finished with value:
162.08432124205248 and parameters: {'n_estimators': 197, 'learning_rate':
```

0.06054618190159289, 'num_leaves': 14, 'max_depth': 32, 'l2_leaf_reg': 14}. Best is trial 5 with value: 162.08432124205248.

Best trial: 5. Best value: 162.084: 23%| | 7/30 [00:07<00:22, 1.03it/s]

[I 2026-05-27 14:06:13,536] Trial 6 finished with value: 173.0064644977526 and parameters: {'n_estimators': 279, 'learning_rate': 0.1657593291820106, 'num_leaves': 20, 'max_depth': 100, 'l2_leaf_reg': 13}. Best is trial 5 with value: 162.08432124205248.

Best trial: 7. Best value: 160.312: 27%| | 8/30 [00:08<00:22, 1.04s/it]

[I 2026-05-27 14:06:14,712] Trial 7 finished with value: 160.3118294470835 and parameters: {'n_estimators': 357, 'learning_rate': 0.023995633159976326, 'num_leaves': 16, 'max_depth': 8, 'l2_leaf_reg': 3}. Best is trial 7 with value: 160.3118294470835.

Best trial: 7. Best value: 160.312: 33%| | 10/30 [00:09<00:13, 1.49it/s]

[I 2026-05-27 14:06:15,351] Trial 8 finished with value: 169.7516925293223 and parameters: {'n_estimators': 124, 'learning_rate': 0.026161113991216598, 'num_leaves': 25, 'max_depth': 91, 'l2_leaf_reg': 7}. Best is trial 7 with value: 160.3118294470835.

[I 2026-05-27 14:06:15,480] Trial 9 finished with value: 201.4917334737444 and parameters: {'n_estimators': 61, 'learning_rate': 0.04423654963211358, 'num_leaves': 7, 'max_depth': 37, 'l2_leaf_reg': 15}. Best is trial 7 with value: 160.3118294470835.

Best trial: 7. Best value: 160.312: 37%| | 11/30 [00:10<00:11, 1.62it/s]

[I 2026-05-27 14:06:15,977] Trial 10 finished with value: 166.82163879092786 and parameters: {'n_estimators': 384, 'learning_rate': 0.021177790996412746, 'num_leaves': 5, 'max_depth': 5, 'l2_leaf_reg': 2}. Best is trial 7 with value: 160.3118294470835.

Best trial: 7. Best value: 160.312: 40%| | 12/30 [00:10<00:10, 1.74it/s]

[I 2026-05-27 14:06:16,452] Trial 11 finished with value: 168.78320295511 and parameters: {'n_estimators': 203, 'learning_rate': 0.10263487468813208, 'num_leaves': 11, 'max_depth': 10, 'l2_leaf_reg': 3}. Best is trial 7 with value: 160.3118294470835.

Best trial: 12. Best value: 159.559: 43%| | 13/30 [00:11<00:10, 1.69it/s]

[I 2026-05-27 14:06:17,086] Trial 12 finished with value: 159.55945262887283 and parameters: {'n_estimators': 216, 'learning_rate': 0.03187840043428948, 'num_leaves': 13, 'max_depth': 44, 'l2_leaf_reg': 7}. Best is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 47%| | 14/30 [00:12<00:10,
1.56it/s]

[I 2026-05-27 14:06:17,834] Trial 13 finished with value:
162.34189153476726 and parameters: {'n_estimators': 397, 'learning_rate':
0.030257790874843916, 'num_leaves': 8, 'max_depth': 10, 'l2_leaf_reg': 6}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 50%| | 15/30 [00:13<00:13,
1.15it/s]

[I 2026-05-27 14:06:19,237] Trial 14 finished with value:
164.1065838157815 and parameters: {'n_estimators': 252, 'learning_rate':
0.020483758168800333, 'num_leaves': 28, 'max_depth': 47, 'l2_leaf_reg': 2}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 53%| | 16/30 [00:14<00:13,
1.01it/s]

[I 2026-05-27 14:06:20,491] Trial 15 finished with value:
165.79090503397822 and parameters: {'n_estimators': 170, 'learning_rate':
0.031897185340152207, 'num_leaves': 38, 'max_depth': 9, 'l2_leaf_reg': 4}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 57%| | 17/30 [00:16<00:14,
1.09s/it]

[I 2026-05-27 14:06:21,840] Trial 16 finished with value:
165.42387577336532 and parameters: {'n_estimators': 310, 'learning_rate':
0.03361693261273962, 'num_leaves': 19, 'max_depth': 56, 'l2_leaf_reg': 9}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 60%| | 18/30 [00:16<00:11,
1.04it/s]

[I 2026-05-27 14:06:22,480] Trial 17 finished with value:
161.59913407445214 and parameters: {'n_estimators': 242, 'learning_rate':
0.024326708183997854, 'num_leaves': 10, 'max_depth': 23, 'l2_leaf_reg': 2}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 63%| | 19/30 [00:17<00:08,
1.23it/s]

[I 2026-05-27 14:06:22,966] Trial 18 finished with value:
165.23075948676174 and parameters: {'n_estimators': 329, 'learning_rate':
0.037674826859908736, 'num_leaves': 6, 'max_depth': 14, 'l2_leaf_reg': 3}. Best
is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 67%| | 20/30 [00:17<00:07,
1.38it/s]

[I 2026-05-27 14:06:23,478] Trial 19 finished with value:
159.58739406813513 and parameters: {'n_estimators': 153, 'learning_rate':

0.05395522753440142, 'num_leaves': 14, 'max_depth': 45, 'l2_leaf_reg': 8}. Best is trial 12 with value: 159.55945262887283.

Best trial: 12. Best value: 159.559: 70%| | 21/30 [00:17<00:05, 1.70it/s]

[I 2026-05-27 14:06:23,744] Trial 20 finished with value: 161.30722892010763 and parameters: {'n_estimators': 110, 'learning_rate': 0.08146124867323389, 'num_leaves': 9, 'max_depth': 52, 'l2_leaf_reg': 9}. Best is trial 12 with value: 159.55945262887283.

Best trial: 21. Best value: 157.47: 73%| | 22/30 [00:18<00:04, 1.81it/s]

[I 2026-05-27 14:06:24,217] Trial 21 finished with value: 157.47027262041885 and parameters: {'n_estimators': 153, 'learning_rate': 0.05457090570146601, 'num_leaves': 14, 'max_depth': 40, 'l2_leaf_reg': 9}. Best is trial 21 with value: 157.47027262041885.

Best trial: 22. Best value: 156.964: 77%| | 23/30 [00:18<00:03, 1.96it/s]

[I 2026-05-27 14:06:24,632] Trial 22 finished with value: 156.96378515119414 and parameters: {'n_estimators': 150, 'learning_rate': 0.05642660979721191, 'num_leaves': 12, 'max_depth': 41, 'l2_leaf_reg': 10}. Best is trial 22 with value: 156.96378515119414.

Best trial: 22. Best value: 156.964: 80%| | 24/30 [00:19<00:02, 2.15it/s]

[I 2026-05-27 14:06:24,986] Trial 23 finished with value: 157.6190756824996 and parameters: {'n_estimators': 129, 'learning_rate': 0.07316453557417973, 'num_leaves': 12, 'max_depth': 66, 'l2_leaf_reg': 11}. Best is trial 22 with value: 156.96378515119414.

Best trial: 22. Best value: 156.964: 83%| | 25/30 [00:19<00:02, 2.00it/s]

[I 2026-05-27 14:06:25,570] Trial 24 finished with value: 165.32655279604626 and parameters: {'n_estimators': 125, 'learning_rate': 0.07963114212810563, 'num_leaves': 22, 'max_depth': 65, 'l2_leaf_reg': 11}. Best is trial 22 with value: 156.96378515119414.

Best trial: 22. Best value: 156.964: 90%| | 27/30 [00:20<00:01, 2.77it/s]

[I 2026-05-27 14:06:25,850] Trial 25 finished with value: 157.37497726250433 and parameters: {'n_estimators': 92, 'learning_rate': 0.0689820853312178, 'num_leaves': 12, 'max_depth': 75, 'l2_leaf_reg': 11}. Best is trial 22 with value: 156.96378515119414.

[I 2026-05-27 14:06:26,042] Trial 26 finished with value: 159.28292317056284 and parameters: {'n_estimators': 91, 'learning_rate': 0.11852673583873038, 'num_leaves': 8, 'max_depth': 76, 'l2_leaf_reg': 20}. Best is trial 22 with value: 156.96378515119414.

Best trial: 22. Best value: 156.964: 93%| | 28/30 [00:20<00:00, 2.86it/s]

[I 2026-05-27 14:06:26,366] Trial 27 finished with value: 165.55218474117433 and parameters: {'n_estimators': 83, 'learning_rate': 0.05191991034116523, 'num_leaves': 17, 'max_depth': 25, 'l2_leaf_reg': 5}. Best is trial 22 with value: 156.96378515119414.

Best trial: 22. Best value: 156.964: 100%| | 30/30 [00:21<00:00, 1.43it/s]

[I 2026-05-27 14:06:26,641] Trial 28 finished with value: 168.51237120674978 and parameters: {'n_estimators': 174, 'learning_rate': 0.10341324226535657, 'num_leaves': 6, 'max_depth': 36, 'l2_leaf_reg': 10}. Best is trial 22 with value: 156.96378515119414.

[I 2026-05-27 14:06:26,815] Trial 29 finished with value: 168.6611493736375 and parameters: {'n_estimators': 68, 'learning_rate': 0.06862947721312534, 'num_leaves': 9, 'max_depth': 19, 'l2_leaf_reg': 5}. Best is trial 22 with value: 156.96378515119414.

```
[15]: LGBMRegressor(force_col_wise=True, learning_rate=0.05642660979721191,
                    loss_function='mae', max_depth=41, n_estimators=150,
                    num_leaves=12, random_state=0, reg_lambda=10, verbose=-1)
```

1.10.8 Evaluation du modèle et prévision

On compare les MAE de cross validation de la RandomizedSearchCV et de celle avec Optuna.

```
[16]: # MAE de cross validation de la random search
print("Best CV mean score - Random search:", -random_search.best_score_)
# Avec optuna
print("Best CV mean score - Optuna:", study.best_value)
```

Best CV mean score - Random search: 160.7246261098385

Best CV mean score - Optuna: 156.96378515119414

Le modèle (ici issu de l'optimisation bayésienne avec Optuna) utilisant le gradient boosting peut être testé sur les 4 derniers mois de données. Sa MAE est la suivante :

```
[17]: from sklearn.metrics import mean_absolute_error

print(f"MAE du modèle : {mean_absolute_error(y_test, optuna_search.
      ↪predict(X_test)):.2f} MWh")
```

MAE du modèle : 150.80 MWh

A comparer avec la MAE du modèle de benchmark simple ! Les variables exogènes et le gradient boosting permettent d'améliorer les performances de prévision à H+48 par rapport au modèle de benchmark simple qui utilise uniquement la consommation à H-48.

Maintenant prévoyons la consommation électrique à H+48 pour les prochaines heures (jusqu'à H+48) et visualisons nos prévisions.

```
[18]: X_train
      # forecast_data["consommation_elec_prevue"] = search.predict(X_forecast)
```

```
[18]:          temperature  vacances \
datetime
2023-01-01 00:00:00+01:00      11.1      True
2023-01-01 01:00:00+01:00      10.8      True
2023-01-01 02:00:00+01:00      10.6      True
2023-01-01 03:00:00+01:00      10.6      True
2023-01-01 04:00:00+01:00      10.5      True
...
2026-01-29 07:00:00+01:00       5.4     False
2026-01-29 08:00:00+01:00       5.1     False
2026-01-29 09:00:00+01:00       4.9     False
2026-01-29 10:00:00+01:00       4.9     False
2026-01-29 11:00:00+01:00       5.2     False
```

```
          consommation_elec_h_minus_48 \
datetime
2023-01-01 00:00:00+01:00          NaN
2023-01-01 01:00:00+01:00          NaN
2023-01-01 02:00:00+01:00          NaN
2023-01-01 03:00:00+01:00          NaN
2023-01-01 04:00:00+01:00          NaN
...
2026-01-29 07:00:00+01:00        3226.0
2026-01-29 08:00:00+01:00        3506.0
2026-01-29 09:00:00+01:00        3658.0
2026-01-29 10:00:00+01:00        3644.0
2026-01-29 11:00:00+01:00        3648.0
```

```
          consommation_elec_mean_week  minute_in_day \
datetime
2023-01-01 00:00:00+01:00          NaN           0
2023-01-01 01:00:00+01:00          NaN          60
2023-01-01 02:00:00+01:00          NaN         120
2023-01-01 03:00:00+01:00          NaN         180
2023-01-01 04:00:00+01:00          NaN         240
...
2026-01-29 07:00:00+01:00        3336.559524          420
2026-01-29 08:00:00+01:00        3335.755952          480
2026-01-29 09:00:00+01:00        3335.029762          540
2026-01-29 10:00:00+01:00        3334.208333          600
2026-01-29 11:00:00+01:00        3333.345238          660
```

```
          day_of_week  day_of_year
datetime
```

2023-01-01 00:00:00+01:00	6	1
2023-01-01 01:00:00+01:00	6	1
2023-01-01 02:00:00+01:00	6	1
2023-01-01 03:00:00+01:00	6	1
2023-01-01 04:00:00+01:00	6	1
...	...	...
2026-01-29 07:00:00+01:00	3	29
2026-01-29 08:00:00+01:00	3	29
2026-01-29 09:00:00+01:00	3	29
2026-01-29 10:00:00+01:00	3	29
2026-01-29 11:00:00+01:00	3	29

[26988 rows x 7 columns]

```
[19]: # Ajout des prévisions du modèle de gradient boosting sur les datasets de test
      ↪ et de prévision
```

```
test_data["consommation_elec_prevue"] = optuna_search.predict(X_test)
forecast_data["consommation_elec_prevue"] = optuna_search.predict(X_forecast)
```

```
[20]: # Visualisation des prévisions à H+48 pour les prochaines heures
```

```
fig = go.Figure()
# Limitons les données réelles sur les 2 dernières semaines pour mieux
  ↪ visualiser les prévisions à H+48
```

```
recent_data = test_data.tail(24 * 7) # 7 derniers jours
```

```
# Réel
```

```
fig.add_trace(go.Scattergl(
    x=recent_data.index,
    y=recent_data["consommation_elec"],
    mode='markers+lines',
    name="Consommation électrique réelle (MWh)",
    marker=dict(color='blue', size=5),
    line=dict(color='blue', dash='dot')
))
```

```
# Prévu sur test_data
```

```
fig.add_trace(go.Scattergl(
    x=recent_data.index,
    y=recent_data["consommation_elec_prevue"],
    mode='markers+lines',
    name="Consommation électrique prévue sur test (MWh)",
    marker=dict(color='orange', size=5),
    line=dict(color='orange', dash='dot')
))
```

```
# Prévisions à H+48
```

```
fig.add_trace(go.Scattergl(
    x=forecast_data.index,
    y=forecast_data["consommation_elec_prevue"],
```

```

mode='markers+lines',
name="Consommation électrique prévue à H+48 (MWh)",
marker=dict(color='red', size=5),
line=dict(color='red', dash='dot')
))
# Libellés de l'axe X avec jour de semaine en français
jours_fr = {
    0: "Lun", 1: "Mar", 2: "Mer", 3: "Jeu",
    4: "Ven", 5: "Sam", 6: "Dim"
}

# Un tick par jour (à minuit) sur la fenêtre affichée
tick_vals = recent_data.index[recent_data.index.hour == 0].append(
    forecast_data.index[forecast_data.index.hour == 0]
).unique().sort_values()

tick_text = [f"{jours_fr[d.weekday()]} {d.strftime('%d/%m')}" for d in
    ↪tick_vals]

fig.update_layout(
    title="Prévision de la consommation électrique à H+48",
    xaxis_title="Date",
    yaxis_title="Consommation électrique (MWh)",
    height=600,
    width=1000
)

fig.update_xaxes(
    tickmode="array",
    tickvals=tick_vals,
    ticktext=tick_text
)
fig.show()

```

1.11 Explicabilité du modèle et importance des regresseurs

Des explications du modèle dans sa globalité - global feature importance - et par prédiction - local feature importance - peuvent être obtenues avec la méthode des *Shapley values*. Elle est mise en oeuvre à l'aide, notamment, de la bibliothèque `shap`.

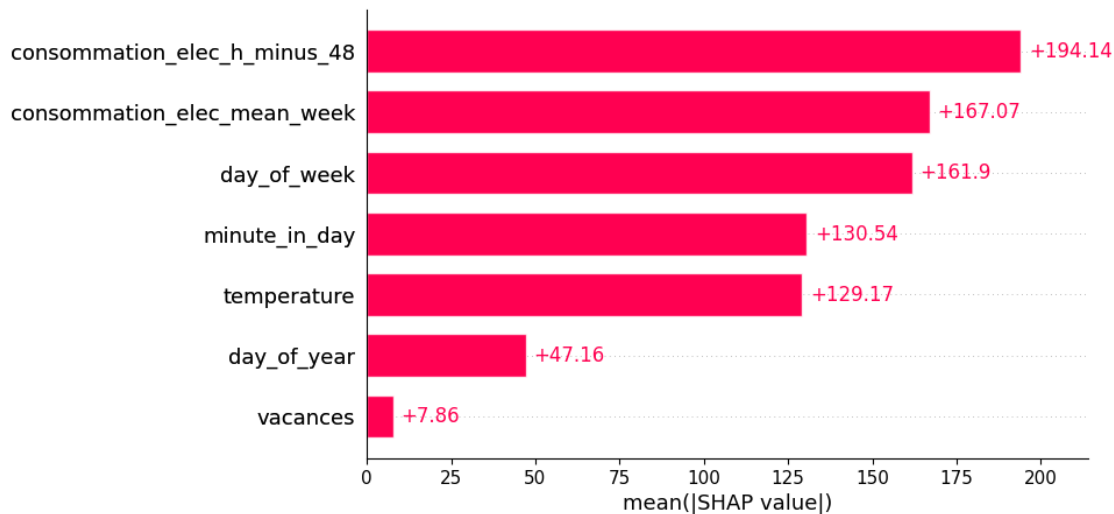
- L'explication globale donne un score d'importance de chaque variable dans le modèle, en moyenne sur l'ensemble des prédictions. On souhaite comprendre le comportement global du modèle appris, donc les données de train sont utilisées pour calculer l'explication globale.

```

[21]: # Prepare the explainer for the best model found by Optuna
from shap import TreeExplainer, plots
explainer = TreeExplainer(optuna_search)

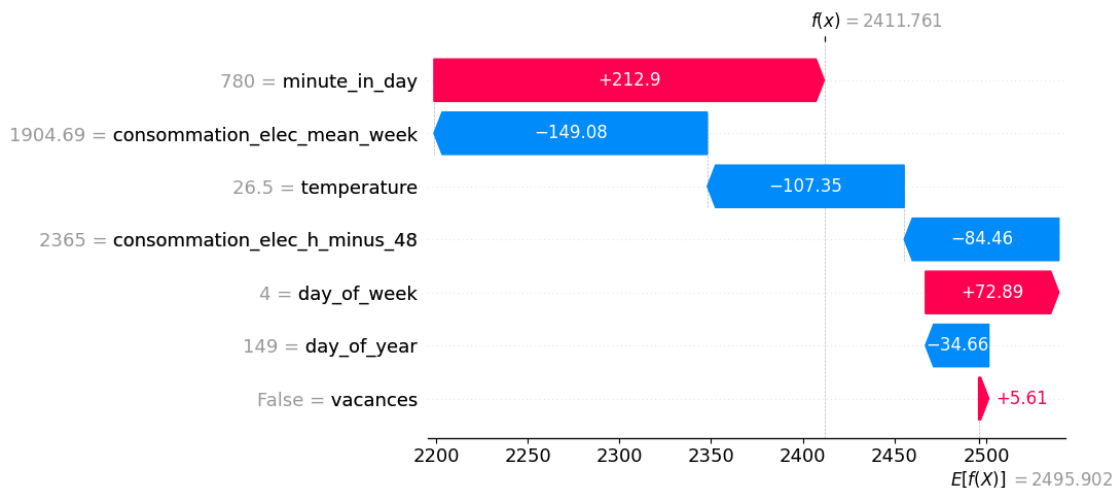
```

```
# Global feature importance
shap_values = explainer(X_train)
plots.bar(shap_values, max_display=None)
```



- L'explication locale donne la contribution estimée de chaque variable à la prédiction pour une observation donnée par rapport à la moyenne. Ici on estime le dernier point à prévoir, dans 48h.

```
[22]: # une observation à expliquer
shap_values = explainer(X_forecast.iloc[[-1]])
plots.waterfall(shap_values[0])
```



D'une moyenne de 2496 MW, on voit les impacts principaux des régresseurs : moyenne de la

dernière semaine, le moment de la journée, la température et le jour de l'année.